

Lesson 4 Structured Note: Complete C Array-Stack Coding Workflow

0. Current Position in the Course

You are now in **Lesson 4** of the stack curriculum.

This lesson belongs to:

Phase 1: Foundations and Array Stack

Before Lesson 4, you learned these ideas:

Lesson	Main focus	What you learned
Lesson 1	Stack basics	A stack follows LIFO: Last In, First Out. Insertion and deletion happen only at the top.
Lesson 2	Array stack representation	An array stack uses <code>size</code> , <code>top</code> , and an array pointer <code>S</code> . An empty array stack has <code>top = -1</code> .
Lesson 3	Array stack operations	You learned <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 4	Complete coding workflow	You now combine everything into one complete C program.

Lesson 4 is not mainly about learning a new data structure. It is about assembling the full array-stack implementation in C.

In simple words:

Lesson 1 gave the idea.

Lesson 2 showed how to store the stack.

Lesson 3 showed the operations.

Lesson 4 puts everything together into a working C program.

1. Lesson 4 Goal

By the end of this lesson, you should be able to write a complete C-style array stack program that includes:

```
create()
display()
push()
pop()
peek()
stackTop()
isEmpty()
isFull()
```

You should also understand:

- Why the stack structure needs `size`, `top`, and `S`.
- Why `create()` uses a pointer to the stack.
- Why `display()` prints from top to bottom.
- Why `push()` must check overflow.
- Why `pop()` must check underflow.
- Why `peek()` uses the formula `top - pos + 1`.
- Which functions change the stack.
- Which functions only read the stack.
- How to dry-run the full program.
- The time and space complexity of each function.

2. Main Idea of Lesson 4

A stack has a rule:

The last value inserted is the first value removed.

This is called:

LIFO: Last In, First Out

In an array stack, values are stored inside an array.

Example:

```
Index: 0   1   2
Value: 10  20  30
```

The newest value is `30`, so it is the top value.

```
Top -> 30
      20
      10
```

The physical array order is:

```
10, 20, 30
```

But the logical stack display order is:

```
30, 20, 10
```

This difference is important.

3. Stack Structure in C

In C, we group the stack information inside a `struct`.

```
struct Stack {
    int size;
    int top;
    int *S;
};
```

Meaning of Each Field

Field	Simple meaning
<code>size</code>	Maximum number of values the stack can store
<code>top</code>	Index of the current top element
<code>S</code>	Pointer to the array where stack values are stored

You can imagine it like this:

```
Stack
├─ size = 5
├─ top  = -1
└─ S    -> array of 5 integers
```

4. What Does `size` Mean?

`size` stores the capacity of the stack.

If:

```
size = 5;
```

Then the array has 5 slots:

```
Index: 0   1   2   3   4
Value: _   _   _   _   _
```

The valid indices are:

```
0, 1, 2, 3, 4
```

The last valid index is:

```
size - 1
```

For size 5:

```
size - 1 = 4
```

So index `4` is the last valid array position.

5. What Does `top` Mean?

`top` stores the index of the current top element.

Example:

```
Index: 0   1   2
Value: 10  20  30
```

Here, the top value is `30`.

30 is stored at index 2.

So:

```
top = 2;
```

Important:

top stores the index, not the value.

Wrong idea:

```
top = 30
```

Correct idea:

```
top = 2  
S[top] = 30
```

6. What Does S Mean?

S is a pointer to the array used by the stack.

```
int *S;
```

Later, memory is allocated using malloc():

```
st->S = (int *)malloc(st->size * sizeof(int));
```

If the stack size is 5, then S points to an array with 5 integer slots.

```
S -> [ ] [ ] [ ] [ ] [ ]
```

7. Required Header Files

A complete C stack program needs these header files:

```
#include <stdio.h>
#include <stdlib.h>
```

Why `stdio.h`?

`stdio.h` is used for input and output functions:

```
printf()
scanf()
```

Why `stdlib.h`?

`stdlib.h` is used for dynamic memory functions:

```
malloc()
free()
exit()
```

8. Empty and Full Conditions

Before writing the functions, remember these two formulas.

Empty Condition

An array stack is empty when:

```
top == -1
```

This means no valid array index is active.

Example:

```
Index: 0   1   2   3   4
Value: _   _   _   _   _
top = -1
```

Important:

`top = -1` does not mean `S[-1]` exists.

Array index `-1` is invalid.

It is only a marker meaning the stack is empty.

Full Condition

An array stack is full when:

```
top == size - 1
```

Example:

```
size = 5  
valid indices = 0, 1, 2, 3, 4
```

The stack is full when:

```
top = 4
```

Because:

```
size - 1 = 5 - 1 = 4
```

9. `create()`: Creating the Stack

The `create()` function builds the stack before we use it.

It does three things:

1. Reads the stack size from the user.
2. Sets `top = -1`.
3. Allocates memory for the array using `malloc()`.

Code

```
void create(struct Stack *st) {  
    printf("Enter size: ");  
    scanf("%d", &st->size);  
  
    st->top = -1;
```

```
st->S = (int *)malloc(st->size * sizeof(int));

if (st->S == NULL) {
    printf("Memory allocation failed\n");
    exit(1);
}
}
```

10. Why Does `create()` Use `struct Stack *st`?

The function receives:

```
struct Stack *st
```

This means `st` is a pointer to the real stack.

`create()` must modify the original stack because it changes:

```
st->size
st->top
st->S
```

If we passed the stack by value, only a copy would change.

So `create()` must receive the stack by address.

In `main()`, we call it like this:

```
create(&st);
```

The `&` means:

Send the address of `st`.

11. Dot Operator vs Arrow Operator

This is a common beginner problem.

Use Dot `.` When You Have a Normal Structure Variable

Example:

```
struct Stack st;  
st.top = -1;
```

Here, `st` is a normal variable.

So we use:

```
st.top
```

Use Arrow `->` When You Have a Pointer to a Structure

Example:

```
void create(struct Stack *st)
```

Here, `st` is a pointer.

So we use:

```
st->top  
st->size  
st->S
```

Simple rule:

Situation	Operator
Normal structure variable	<code>.</code>
Pointer to structure	<code>-></code>

12. Meaning of `malloc(st->size * sizeof(int))`

This line creates the stack array dynamically:

```
st->S = (int *)malloc(st->size * sizeof(int));
```

Break it down:

```
st->size
```

This is the number of integers we want.

```
sizeof(int)
```

This is the memory needed for one integer.

```
st->size * sizeof(int)
```

This gives the total memory needed for all stack elements.

Example:

If:

```
st->size = 5
```

Then:

```
malloc(5 * sizeof(int))
```

This allocates memory for 5 integers.

The cast:

```
(int *)
```

means:

Treat the memory address returned by `malloc()` as a pointer to integer.

13. Why Check `malloc()` Result?

Sometimes memory allocation can fail.

If `malloc()` fails, it returns `NULL`.

So this check is useful:

```
if (st->S == NULL) {
    printf("Memory allocation failed\n");
    exit(1);
}
```

This prevents the program from using invalid memory.

For beginner programs, some examples omit this check, but it is safer to include it.

14. `display()`: Showing the Stack

The `display()` function prints all active stack elements.

Code

```
void display(struct Stack st) {
    int i;

    for (i = st.top; i >= 0; i--) {
        printf("%d ", st.S[i]);
    }

    printf("\n");
}
```

15. Why Does `display()` Print from `top` Down to `0`?

The physical array stores values from left to right.

Example:

```
Index: 0   1   2
Value: 10  20  30
top = 2
```

But the logical stack is viewed from top to bottom:

```
Top -> 30
      20
      10
```

So `display()` should print:

```
30 20 10
```

That is why the loop starts at `top` and moves down to `0`:

```
for (i = st.top; i >= 0; i--)
```

16. Does `display()` Change the Stack?

No.

`display()` only reads the stack.

So it can receive the stack by value:

```
void display(struct Stack st)
```

It does not need:

```
struct Stack *st
```

because it does not modify the original stack.

17. `push()`: Insert a Value at the Top

`push()` inserts a new value at the top of the stack.

Main Idea

A successful `push()` does two things:

1. Increase `top` by 1.
2. Store the new value at `S[top]`.

In code:

```
st->top++;  
st->S[st->top] = x;
```

Code

```
void push(struct Stack *st, int x) {  
    if (st->top == st->size - 1) {  
        printf("Stack Overflow\n");  
    } else {  
        st->top++;  
        st->S[st->top] = x;  
    }  
}
```

18. Why Does `push()` Check Overflow?

Before inserting a value, we must check whether the stack is already full.

The stack is full when:

```
st->top == st->size - 1
```

If the stack is full and we still push, the program may write outside the valid array memory.

That is an error.

This error is called:

Stack Overflow

Example:

```
size = 5  
top = 4
```

The valid indices are:

```
0, 1, 2, 3, 4
```

Since `top = 4`, the stack is already using the last valid index.

Another push would cause overflow.

19. Why Does `push()` Use `struct Stack *st`?

`push()` changes the original stack.

It changes:

```
st->top  
st->S[st->top]
```

So it must receive the stack by pointer:

```
void push(struct Stack *st, int x)
```

In `main()`, we call it like this:

```
push(&st, 10);
```

20. `push()` Dry Run

Given:

```
size = 5  
top = 1
```

Current stack:

```
Index: 0   1   2   3   4  
Value: 10  20  _   _   _  
top = 1
```

Now call:

```
push(&st, 30);
```

Step 1: Check Full Condition

```
top == size - 1  
1 == 5 - 1  
1 == 4
```

This is false.

So the stack is not full.

Step 2: Increase `top`

```
top: 1 -> 2
```

Step 3: Store the New Value

```
S[2] = 30
```

Final stack:

```
Index: 0   1   2   3   4  
Value: 10  20  30  _   _  
top = 2
```

Logical stack:

```
Top -> 30
      20
      10
```

21. `pop()` : Remove the Top Value

`pop()` removes and returns the top element.

Main Idea

A successful `pop()` does three things:

1. Save `S[top]`.
2. Decrease `top` by 1.
3. Return the saved value.

In code:

```
x = st->S[st->top];
st->top--;
return x;
```

Code

```
int pop(struct Stack *st) {
    int x = -1;

    if (st->top == -1) {
        printf("Stack Underflow\n");
    } else {
        x = st->S[st->top];
        st->top--;
    }

    return x;
}
```


22. Why Does `pop()` Check Underflow?

Before removing a value, we must check whether the stack is empty.

The stack is empty when:

```
st->top == -1
```

If the stack is empty, there is no value to remove.

This error is called:

Stack Underflow

Example:

```
top = -1
```

Now if we call:

```
pop(&st);
```

There is no top value.

So the function prints:

```
Stack Underflow
```

and returns `-1`.

23. Why Is `x` Initialized to `-1` in `pop()`?

In `pop()` we write:

```
int x = -1;
```

`x` stores the value to be returned.

If the stack is empty, there is no valid value to return.

So `-1` is used as an error value.

If the pop is successful, `x` is replaced with the real top value:

```
x = st->S[st->top];
```

Important note:

Using `-1` as an error value is simple, but it has a limitation.

If `-1` is a valid stack value, then the caller cannot easily know whether `-1` means an error or a real popped value.

For beginner lessons, this simple method is acceptable.

24. Why Does `pop()` Use `struct Stack *st`?

`pop()` changes the stack because it decreases `top`.

So it must receive the stack by pointer:

```
int pop(struct Stack *st)
```

In `main()`, we call it like this:

```
pop(&st);
```

25. `pop()` Dry Run

Given:

```
Index: 0   1   2
Value: 10  20  30
top = 2
```

Logical stack:

```
Top -> 30
      20
      10
```

Now call:

```
pop(&st);
```

Step 1: Check Empty Condition

```
top == -1
2 == -1
```

This is false.

So the stack is not empty.

Step 2: Save the Top Value

```
x = S[top]
x = S[2]
x = 30
```

Step 3: Decrease `top`

```
top: 2 -> 1
```

Step 4: Return Saved Value

```
pop() returns 30
```

Final logical stack:

```
Top -> 20
      10
```

26. Physical Array vs Logical Stack After `pop()`

After the pop, the physical array may still look like this:

```
Index: 0   1   2
Value: 10  20  30
top = 1
```

But the logical stack is only:

```
Top -> 20
      10
```

Why?

Because `top = 1`.

So only indices `0` and `1` are active.

Index `2` is ignored.

Important idea:

A popped value may still physically remain in memory, but it is no longer logically part of the stack.

27. `peek()` : Read a Position from the Top

`peek(pos)` reads a value at a position from the top without deleting it.

Example:

```
Top -> 40   position 1
      30   position 2
      20   position 3
      10   position 4
```

So:

```
peek(1) returns 40
peek(2) returns 30
peek(3) returns 20
peek(4) returns 10
```

`peek()` does not remove anything.

28. Why Does `peek()` Need a Formula?

The user gives a position from the top.

But the array uses indices from the left side.

So we must convert:

```
position from top -> array index
```

The formula is:

```
index = top - pos + 1;
```

This is one of the most important formulas in array stack operations.

29. `peek()` Code

```
int peek(struct Stack st, int pos) {
    int index = st.top - pos + 1;

    if (pos <= 0 || index < 0 || index > st.top) {
        printf("Invalid Index\n");
        return -1;
    }

    return st.S[index];
}
```

This version checks:

```
pos <= 0
```

because valid positions start from `1`, not `0`.

It also checks:

```
index < 0
```

because negative array indices are invalid.

It also checks:

```
index > st.top
```

because we should not read beyond the active part of the stack.

30. Why Can `peek()` Receive Stack by Value?

`peek()` only reads the stack.

It does not change:

```
top  
s  
size
```

So it can receive a copy:

```
int peek(struct Stack st, int pos)
```

It does not need a pointer.

31. `peek()` Dry Run

Given:

```
Index: 0   1   2   3
Value: 10  20  30  40
top = 3
```

Logical stack:

```
Top -> 40   position 1
        30   position 2
        20   position 3
        10   position 4
```

Now call:

```
peek(st, 2);
```

Use the formula:

```
index = top - pos + 1
index = 3 - 2 + 1
index = 2
```

So:

```
peek(2) returns S[2]
```

Since:

```
S[2] = 30
```

Final answer:

```
peek(2) returns 30
```

The stack does not change.

32. More peek() Examples

Given:

```
Index: 0   1   2   3
Value: 10  20  30  40
top = 3
```

Logical stack:

```
Top -> 40   position 1
        30   position 2
        20   position 3
        10   position 4
```

peek(1)

```
index = 3 - 1 + 1
index = 3
```

So:

```
peek(1) returns S[3] = 40
```

peek(2)

```
index = 3 - 2 + 1
index = 2
```

So:

```
peek(2) returns S[2] = 30
```

peek(3)

```
index = 3 - 3 + 1
index = 1
```

So:


```
peek(3) returns S[1] = 20
```

```
peek(4)
```

```
index = 3 - 4 + 1  
index = 0
```

So:

```
peek(4) returns S[0] = 10
```

```
peek(5)
```

```
index = 3 - 5 + 1  
index = -1
```

Index `-1` is invalid.

So the function prints:

```
Invalid Index
```

and returns:

```
-1
```

33. `stackTop()`: Read the Current Top Value

`stackTop()` returns the current top value without deleting it.

It is similar to:

```
peek(st, 1)
```

But it is simpler because it only reads the top element.

Code

```
int stackTop(struct Stack st) {  
    if (st.top == -1) {  
        return -1;  
    }  
  
    return st.S[st.top];  
}
```

34. `stackTop()` Example

Given:

```
Index: 0   1   2  
Value: 10  20  30  
top = 2
```

Call:

```
stackTop(st);
```

Since:

```
S[top] = S[2] = 30
```

The function returns:

```
30
```

The stack does not change.

35. Difference Between `pop()` and `stackTop()`

Operation	Returns top value?	Removes top value?	Changes <code>top</code> ?
<code>pop()</code>	Yes	Yes	Yes

Operation	Returns top value?	Removes top value?	Changes <code>top</code> ?
<code>stackTop()</code>	Yes	No	No

Example stack:

```
Top -> 30
      20
      10
```

If we call:

```
stackTop(st);
```

It returns:

```
30
```

Stack remains:

```
Top -> 30
      20
      10
```

If we call:

```
pop(&st);
```

It returns:

```
30
```

Stack becomes:

```
Top -> 20
      10
```

36. isEmpty(): Check Whether Stack Is Empty

isEmpty() checks whether the stack has no active values.

Code

```
int isEmpty(struct Stack st) {  
    return st.top == -1;  
}
```

It returns:

Return value	Meaning
1	Stack is empty
0	Stack is not empty

Example:

```
top = -1
```

Then:

```
isEmpty(st) returns 1
```

Example:

```
top = 2
```

Then:

```
isEmpty(st) returns 0
```

isEmpty() only reads, so it receives the stack by value.

37. isFull(): Check Whether Stack Is Full

isFull() checks whether the stack has no free space left.

Code

```
int isFull(struct Stack st) {  
    return st.top == st.size - 1;  
}
```

It returns:

Return value	Meaning
1	Stack is full
0	Stack is not full

Example:

```
size = 5  
top = 4
```

Check:

```
top == size - 1  
4 == 5 - 1  
4 == 4
```

This is true.

So:

```
isFull(st) returns 1
```

Example:

```
size = 5  
top = 2
```

Check:

```
top == size - 1
2 == 5 - 1
2 == 4
```

This is false.

So:

```
isFull(st) returns 0
```

38. Functions That Modify the Stack vs Functions That Only Read

Some functions change the stack.

Some functions only read the stack.

Function	Changes stack?	Passing style	Why
<code>create()</code>	Yes	Pass by pointer	It sets <code>size</code> , <code>top</code> , and <code>S</code> .
<code>push()</code>	Yes	Pass by pointer	It changes <code>top</code> and stores a value.
<code>pop()</code>	Yes	Pass by pointer	It changes <code>top</code> .
<code>display()</code>	No	Pass by value	It only prints values.
<code>peek()</code>	No	Pass by value	It only reads one value.
<code>stackTop()</code>	No	Pass by value	It only reads the top value.
<code>isEmpty()</code>	No	Pass by value	It only checks <code>top</code> .
<code>isFull()</code>	No	Pass by value	It only checks <code>top</code> and <code>size</code> .

Simple rule:

If a function changes the stack, pass the stack by pointer.
If a function only reads the stack, pass the stack by value.

39. Complete C Program for Lesson 4

```
#include <stdio.h>
#include <stdlib.h>
```

```

struct Stack {
    int size;
    int top;
    int *S;
};

void create(struct Stack *st) {
    printf("Enter size: ");
    scanf("%d", &st->size);

    st->top = -1;
    st->S = (int *)malloc(st->size * sizeof(int));

    if (st->S == NULL) {
        printf("Memory allocation failed\n");
        exit(1);
    }
}

void display(struct Stack st) {
    int i;

    for (i = st.top; i >= 0; i--) {
        printf("%d ", st.S[i]);
    }

    printf("\n");
}

void push(struct Stack *st, int x) {
    if (st->top == st->size - 1) {
        printf("Stack Overflow\n");
    } else {
        st->top++;
        st->S[st->top] = x;
    }
}

int pop(struct Stack *st) {
    int x = -1;

    if (st->top == -1) {
        printf("Stack Underflow\n");
    } else {
        x = st->S[st->top];
        st->top--;
    }
}

```

```

        return x;
    }

    int peek(struct Stack st, int pos) {
        int index = st.top - pos + 1;

        if (pos <= 0 || index < 0 || index > st.top) {
            printf("Invalid Index\n");
            return -1;
        }

        return st.S[index];
    }

    int stackTop(struct Stack st) {
        if (st.top == -1) {
            return -1;
        }

        return st.S[st.top];
    }

    int isEmpty(struct Stack st) {
        return st.top == -1;
    }

    int isFull(struct Stack st) {
        return st.top == st.size - 1;
    }

    int main() {
        struct Stack st;

        create(&st);

        push(&st, 10);
        push(&st, 20);
        push(&st, 30);
        push(&st, 40);

        printf("Stack elements: ");
        display(st);

        printf("Popped value: %d\n", pop(&st));

        printf("Stack after pop: ");
        display(st);
    }

```



```
    printf("Peek position 2: %d\n", peek(st, 2));

    printf("Stack top: %d\n", stackTop(st));

    printf("Is empty? %d\n", isEmpty(st));
    printf("Is full? %d\n", isFull(st));

    free(st.S);

    return 0;
}
```

40. Explanation of the Complete Program

Step 1: Include Header Files

```
#include <stdio.h>
#include <stdlib.h>
```

`stdio.h` gives us:

```
printf()
scanf()
```

`stdlib.h` gives us:

```
malloc()
free()
exit()
```

Step 2: Define the Stack Structure

```
struct Stack {
    int size;
    int top;
    int *S;
};
```

This structure stores all stack information.

Step 3: Create the Stack

```
struct Stack st;  
create(&st);
```

This creates a stack variable and initializes it.

If the user enters , then:

```
size = 5  
top = -1  
S points to an array of 5 integers
```

Step 4: Push Values

```
push(&st, 10);  
push(&st, 20);  
push(&st, 30);  
push(&st, 40);
```

After these operations:

```
Index: 0   1   2   3   4  
Value: 10  20  30  40  _  
top = 3
```

Logical stack:

```
Top -> 40  
      30  
      20  
      10
```

Step 5: Display the Stack

```
printf("Stack elements: ");  
display(st);
```

Output:

```
Stack elements: 40 30 20 10
```

Step 6: Pop One Value

```
printf("Popped value: %d\n", pop(&st));
```

The top value is 40, so pop() returns 40.

Now:

```
top = 2
```

Logical stack:

```
Top -> 30  
      20  
      10
```

Step 7: Display After Pop

```
printf("Stack after pop: ");  
display(st);
```

Output:

```
Stack after pop: 30 20 10
```

Step 8: Peek Position 2

```
printf("Peek position 2: %d\n", peek(st, 2));
```

Current logical stack:

```
Top -> 30    position 1
        20    position 2
        10    position 3
```

So:

```
peek(2) returns 20
```

Step 9: Read Stack Top

```
printf("Stack top: %d\n", stackTop(st));
```

Current top value is 30.

So output:

```
Stack top: 30
```

Step 10: Check Empty and Full

```
printf("Is empty? %d\n", isEmpty(st));
printf("Is full? %d\n", isFull(st));
```

Current state:

```
size = 5
top = 2
```

Check empty:

```
top == -1  
2 == -1  
false
```

So:

```
isEmpty(st) returns 0
```

Check full:

```
top == size - 1  
2 == 5 - 1  
2 == 4  
false
```

So:

```
isFull(st) returns 0
```

Step 11: Free Memory

```
free(st.S);
```

Because the stack array was created using `malloc()`, it should be released using `free()`.

This prevents memory leaks.

41. Expected Output Example

Assume the user enters:

```
5
```

A possible output is:

```
Enter size: 5
Stack elements: 40 30 20 10
Popped value: 40
Stack after pop: 30 20 10
Peek position 2: 20
Stack top: 30
Is empty? 0
Is full? 0
```

42. Full Dry Run

Assume the user enters:

5

Initial state after `create(&st)`:

```
size = 5
top = -1
S = array of 5 integers
```

Array:

```
Index: 0   1   2   3   4
Value: _   _   _   _   _
top = -1
```

Operation 1: `push(&st, 10)`

Check full:

```
top == size - 1
-1 == 4
false
```

Increase top:

```
top: -1 -> 0
```

Store value:

```
S[0] = 10
```

Array:

```
Index: 0  1  2  3  4
Value: 10 -  -  -  -
top = 0
```

Operation 2: `push(&st, 20)`

Check full:

```
0 == 4
false
```

Increase top:

```
top: 0 -> 1
```

Store value:

```
S[1] = 20
```

Array:

```
Index: 0  1  2  3  4
Value: 10 20 -  -  -
top = 1
```

Operation 3: `push(&st, 30)`

Check full:

```
1 == 4  
false
```

Increase top:

```
top: 1 -> 2
```

Store value:

```
S[2] = 30
```

Array:

```
Index: 0   1   2   3   4  
Value: 10  20  30  _  _  
top = 2
```

Operation 4: `push(&st, 40)`

Check full:

```
2 == 4  
false
```

Increase top:

```
top: 2 -> 3
```

Store value:

```
S[3] = 40
```

Array:


```
Index: 0   1   2   3   4
Value: 10  20  30  40  _
top = 3
```

Logical stack:

```
Top -> 40
      30
      20
      10
```

Operation 5: `display(st)`

The loop starts at `top = 3` and moves down to `0`.

It prints:

```
40 30 20 10
```

Operation 6: `pop(&st)`

Check empty:

```
top == -1
3 == -1
false
```

Save top value:

```
x = S[3]
x = 40
```

Decrease top:

```
top: 3 -> 2
```

Return:

40

Logical stack now:

```
Top -> 30
      20
      10
```

Physical array may still be:

```
Index: 0  1  2  3  4
Value: 10 20 30 40 _
top = 2
```

But index 3 is ignored.

Operation 7: `display(st)` Again

Now `top = 2`.

The function prints from index 2 down to 0.

Output:

30 20 10

Operation 8: `peek(st, 2)`

Current logical stack:

```
Top -> 30   position 1
      20   position 2
      10   position 3
```

Use formula:

```
index = top - pos + 1  
index = 2 - 2 + 1  
index = 1
```

So:

```
peek(2) returns S[1]
```

Since:

```
S[1] = 20
```

Answer:

```
20
```

Operation 9: `stackTop(st)`

Current top:

```
top = 2
```

So:

```
S[top] = S[2] = 30
```

Answer:

```
30
```

Operation 10: `isEmpty(st)`

Check:

```
top == -1
2 == -1
false
```

So:

```
isEmpty(st) returns 0
```

Operation 11: `isFull(st)`

Check:

```
top == size - 1
2 == 5 - 1
2 == 4
false
```

So:

```
isFull(st) returns 0
```

43. Dry Run Summary Table

Step	Operation	Top before	Top after	Return/output	Active stack
0	Start	-1	-1	—	Empty
1	<code>push(10)</code>	-1	0	—	10
2	<code>push(20)</code>	0	1	—	10, 20
3	<code>push(30)</code>	1	2	—	10, 20, 30
4	<code>push(40)</code>	2	3	—	10, 20, 30, 40
5	<code>display()</code>	3	3	40 30 20 10	10, 20, 30, 40
6	<code>pop()</code>	3	2	returns 40	10, 20, 30
7	<code>display()</code>	2	2	30 20 10	10, 20, 30
8	<code>peek(2)</code>	2	2	returns 20	10, 20, 30

Step	Operation	Top before	Top after	Return/output	Active stack
9	stackTop()	2	2	returns 30	10, 20, 30
10	isEmpty()	2	2	returns 0	10, 20, 30
11	isFull()	2	2	returns 0	10, 20, 30

44. Overflow Test

Assume:

```
size = 5
top = 4
```

Current stack:

```
Index: 0  1  2  3  4
Value: 10 20 30 40 50
top = 4
```

Now call:

```
push(&st, 60);
```

Check full condition:

```
top == size - 1
4 == 5 - 1
4 == 4
true
```

So the function prints:

```
Stack Overflow
```

No value is inserted.

top stays 4.

45. Underflow Test

Assume:

```
top = -1
```

The stack is empty.

Now call:

```
pop(&st);
```

Check empty condition:

```
top == -1  
-1 == -1  
true
```

So the function prints:

```
Stack Underflow
```

and returns:

```
-1
```

`top` stays `-1`.

46. Invalid Peek Test

Assume:

```
Index: 0   1   2  
Value: 10  20  30  
top = 2
```

Logical stack:

```
Top -> 30    position 1
        20    position 2
        10    position 3
```

Now call:

```
peek(st, 4);
```

Formula:

```
index = top - pos + 1
index = 2 - 4 + 1
index = -1
```

Index `-1` is invalid.

So the function prints:

```
Invalid Index
```

and returns:

```
-1
```

47. Time Complexity

Function	Time complexity	Reason
<code>create()</code>	$O(1)$ control work	It reads size and sets values.
<code>display()</code>	$O(n)$	It prints each active stack value.
<code>push()</code>	$O(1)$	It checks full, increases <code>top</code> , and stores one value.
<code>pop()</code>	$O(1)$	It checks empty, reads one value, and decreases <code>top</code> .
<code>peek()</code>	$O(1)$	It calculates an index and reads one value directly.
<code>stackTop()</code>	$O(1)$	It directly reads <code>S[top]</code> .
<code>isEmpty()</code>	$O(1)$	It checks <code>top == -1</code> .

Function	Time complexity	Reason
<code>isFull()</code>	$O(1)$	It checks <code>top == size - 1</code> .

Important:

`display()` is $O(n)$ because it may print all stack elements.

Most other operations are $O(1)$ because they work directly with `top`.

They do not scan the full array.

48. Space Complexity

Stack Storage

The stack array stores up to `n` elements.

So stack storage is:

$O(n)$

`create()`

`create()` allocates the stack array.

So it creates:

$O(n)$ stack storage

`push()`, `pop()`, `peek()`, `stackTop()`, `isEmpty()`, `isFull()`

These functions only use a small fixed number of variables.

So their auxiliary space is:

$O(1)$

`display()`

`display()` uses only a loop variable.

So its auxiliary space is:

$O(1)$

Even though its time is $O(n)$, its extra space is $O(1)$.

49. Complexity Summary Table

Function	Time	Auxiliary space
<code>create()</code>	$O(1)$ control	$O(n)$ storage allocation
<code>display()</code>	$O(n)$	$O(1)$
<code>push()</code>	$O(1)$	$O(1)$
<code>pop()</code>	$O(1)$	$O(1)$
<code>peek()</code>	$O(1)$	$O(1)$
<code>stackTop()</code>	$O(1)$	$O(1)$
<code>isEmpty()</code>	$O(1)$	$O(1)$
<code>isFull()</code>	$O(1)$	$O(1)$

50. Common Beginner Mistakes

Mistake 1: Forgetting to Initialize `top`

Wrong:

```
st.S = (int *)malloc(st.size * sizeof(int));
```

but forgetting:

```
st.top = -1;
```

Problem:

The program may treat garbage memory as the top value.

Correct:

```
st.top = -1;
```

Mistake 2: Starting Empty Stack with `top = 0`

Wrong:

```
top = 0;
```

Problem:

This makes the program think index `0` is already active.

Correct:

```
top = -1;
```

Then the first push changes:

```
top: -1 -> 0
```

Mistake 3: Forgetting Overflow Check Before `push()`

Wrong:

```
st->top++;  
st->S[st->top] = x;
```

Problem:

If the stack is full, this can write outside the array.

Correct:

```
if (st->top == st->size - 1) {  
    printf("Stack Overflow\n");  
} else {  
    st->top++;  
}
```

```
    st->S[st->top] = x;
}
```

Mistake 4: Forgetting Underflow Check Before `pop()`

Wrong:

```
x = st->S[st->top];
st->top--;
```

Problem:

If `top = -1`, this tries to access:

```
S[-1]
```

That is invalid.

Correct:

```
if (st->top == -1) {
    printf("Stack Underflow\n");
} else {
    x = st->S[st->top];
    st->top--;
}
```

Mistake 5: Printing from `0` to `top`

If you print from `0` to `top`, you get physical array order.

Example:

```
10 20 30 40
```

But stack display should usually show the top first:

```
40 30 20 10
```

Correct loop:

```
for (i = st.top; i >= 0; i--)
```

Mistake 6: Wrong Peek Formula

Wrong:

```
index = top - pos;
```

Correct:

```
index = top - pos + 1;
```

The `+ 1` is necessary.

Example:

```
top = 3  
pos = 1
```

Correct formula:

```
index = 3 - 1 + 1  
index = 3
```

So `peek(1)` correctly returns the top element.

Without `+ 1`:

```
index = 3 - 1  
index = 2
```

That would incorrectly return the second element from the top.

Mistake 7: Thinking `peek()` Removes Data

Wrong idea:

```
peek() removes a value.
```

Correct idea:

```
peek() only reads a value.  
pop() removes a value.
```

Mistake 8: Confusing `stackTop()` and `pop()`

`stackTop()` reads the top value.

`pop()` removes and returns the top value.

Function	Reads top?	Removes top?
<code>stackTop()</code>	Yes	No
<code>pop()</code>	Yes	Yes

Mistake 9: Forgetting `&` When Calling Functions That Modify the Stack

Wrong:

```
push(st, 10);  
pop(st);
```

Correct:

```
push(&st, 10);  
pop(&st);
```

Why?

Because `push()` and `pop()` need the address of the original stack.

Mistake 10: Using `.` Instead of `->`

Inside this function:

```
void push(struct Stack *st, int x)
```

`st` is a pointer.

Wrong:

```
st.top
```

Correct:

```
st->top
```

Mistake 11: Forgetting to Free Memory

Because memory was allocated using:

```
malloc()
```

it should be released using:

```
free(st.S);
```

If we forget this, the program may leak memory.

Mistake 12: Mixing Field Names

Some examples use:

```
int *S;
```

Some examples use:

```
int *arr;
```

Both are possible, but you must be consistent.

If your structure says:

```
int *S;
```

then all functions must use:

```
st.S  
st->S
```

Do not suddenly write:

```
st.arr  
st->arr
```

unless your structure actually has a field named `arr`.

51. Important Formulas and Rules

Empty Formula

```
top == -1
```

Full Formula

```
top == size - 1
```

Push Rule

```
top++;  
S[top] = x;
```

Pop Rule

```
x = S[top];  
top--;  
return x;
```

Peek Formula

```
index = top - pos + 1;
```

Stack Top Rule

```
return S[top];
```

Display Rule

```
for (i = top; i >= 0; i--)
```

Number of Active Elements

When the stack is not empty:

```
number of active elements = top + 1;
```

Example:

```
top = 2
```

Then active elements:

```
2 + 1 = 3
```

52. Short Glossary

Term	Simple meaning
Stack	A data structure that follows LIFO

Term	Simple meaning
LIFO	Last In, First Out
Array stack	A stack implemented using an array
<code>size</code>	Maximum number of elements the stack can store
<code>top</code>	Index of the current top element
<code>S</code>	Pointer to the array storage
<code>struct</code>	A C container that groups related variables
<code>malloc()</code>	Function used to allocate heap memory in C
<code>free()</code>	Function used to release heap memory in C
<code>create()</code>	Function that initializes the stack
<code>display()</code>	Function that prints stack elements from top to bottom
<code>push(x)</code>	Insert <code>x</code> at the top
<code>pop()</code>	Remove and return the top value
<code>peek(pos)</code>	Read a value at a position from the top without deleting it
<code>stackTop()</code>	Read only the current top element
<code>isEmpty()</code>	Check whether the stack is empty
<code>isFull()</code>	Check whether the stack is full
Overflow	Trying to push into a full stack
Underflow	Trying to pop from an empty stack
Pass by pointer	Passing an address so the function can modify the original stack
Pass by value	Passing a copy, useful when the function only reads
Logical stack	Stack as seen by stack rules, from top to bottom
Physical array	Actual memory layout in the array
$O(1)$	Constant time
$O(n)$	Time or space grows with the number of elements

53. Checkpoint Questions

Try to answer these without looking at the answers.

1. What is the goal of Lesson 4?
 2. What are the three fields inside `struct Stack`?
 3. What does `size` store?
 4. What does `top` store?
 5. What does `S` store?
 6. Why is `top` initialized to `-1`?
 7. What is the empty condition?
 8. What is the full condition?
 9. What does `create()` do?
 10. Why does `create()` receive `struct Stack *st`?
 11. What does `malloc(st->size * sizeof(int))` do?
 12. Why should we check whether `malloc()` returns `NULL`?
 13. What does `display()` do?
 14. Why does `display()` print from `top` down to `0`?
 15. What does `push()` do?
 16. Why does `push()` check overflow?
 17. What does `pop()` do?
 18. Why does `pop()` check underflow?
 19. What does `peek(pos)` do?
 20. What is the formula for `peek(pos)`?
 21. What does `stackTop()` do?
 22. What is the difference between `pop()` and `stackTop()`?
 23. What does `isEmpty()` return when the stack is empty?
 24. What does `isFull()` return when the stack is full?
 25. Which functions change the stack?
 26. Which functions only read the stack?
 27. Why do `push()` and `pop()` use pointers?
 28. Why can `peek()` and `display()` receive the stack by value?
 29. What is the time complexity of `push()`?
 30. What is the time complexity of `display()`?
 31. Why should we call `free(st.S)` at the end?
 32. What is the difference between physical array content and logical stack content after `pop()`?
-

54. Checkpoint Answers

1. Lesson 4 teaches how to combine all array-stack functions into one complete C program.
2. The three fields are `size`, `top`, and `S`.
3. `size` stores the maximum stack capacity.
4. `top` stores the index of the current top element.
5. `S` stores the address of the array used for stack storage.

6. `top` is initialized to `-1` because no valid array index is active in an empty stack.
7. The empty condition is `top == -1`.
8. The full condition is `top == size - 1`.
9. `create()` reads the size, sets `top = -1`, and allocates memory.
10. `create()` receives `struct Stack *st` because it changes the original stack.
11. It allocates enough heap memory for `st->size` integers.
12. We check for `NULL` to detect memory allocation failure.
13. `display()` prints the active stack elements.
14. It prints from `top` down to `0` because stack display should show the top first.
15. `push()` inserts a new value at the top.
16. `push()` checks overflow to avoid inserting into a full stack.
17. `pop()` removes and returns the top value.
18. `pop()` checks underflow to avoid removing from an empty stack.
19. `peek(pos)` reads a value at a position from the top without deleting it.
20. The formula is `index = top - pos + 1`.
21. `stackTop()` reads the current top value without deleting it.
22. `pop()` removes the top value, but `stackTop()` only reads it.
23. `isEmpty()` returns `1` when the stack is empty.
24. `isFull()` returns `1` when the stack is full.
25. `create()`, `push()`, and `pop()` change the stack.
26. `display()`, `peek()`, `stackTop()`, `isEmpty()`, and `isFull()` only read the stack.
27. `push()` and `pop()` use pointers because they must modify the original stack.
28. `peek()` and `display()` receive by value because they only read from the stack.
29. `push()` is $O(1)$.
30. `display()` is $O(n)$.
31. We call `free(st.S)` to release heap memory allocated by `malloc()`.
32. After `pop()`, the removed value may still physically exist in memory, but it is ignored because `top` moved down.

55. Practice Tasks

Task 1: Write the Structure

Write the stack structure from memory.

Expected answer:

```
struct Stack {
    int size;
    int top;
    int *S;
};
```

Task 2: Write `create()`

Write a function that:

1. Asks the user for size.
 2. Sets `top = -1`.
 3. Allocates memory using `malloc()`.
-

Task 3: Write `display()`

Write a function that prints the stack from `top` down to `0`.

Task 4: Write `push()`

Write a function that:

1. Checks overflow.
 2. Increases `top`.
 3. Stores the new value.
-

Task 5: Write `pop()`

Write a function that:

1. Checks underflow.
 2. Saves `S[top]`.
 3. Decreases `top`.
 4. Returns the saved value.
-

Task 6: Write `peek()`

Write a function that uses:

```
index = top - pos + 1;
```

and returns the value at that index.

Task 7: Trace This Sequence

Start with:

```
size = 5  
top = -1
```

Perform:

```
push(&st, 10);  
push(&st, 20);  
push(&st, 30);  
pop(&st);  
push(&st, 40);  
stackTop(st);
```

Write:

- `top` before each operation.
 - `top` after each operation.
 - Return value, if any.
 - Active stack after each operation.
-

Task 8: Test Overflow

Create a stack of size `3`.

Then run:

```
push(&st, 10);  
push(&st, 20);  
push(&st, 30);  
push(&st, 40);
```

Expected result:

The fourth push should print:

```
Stack Overflow
```

Task 9: Test Underflow

Create an empty stack.

Then call:

```
pop(&st);
```

Expected result:

```
Stack Underflow
```

Task 10: Test Invalid Peek

Given:

```
Index: 0   1   2  
Value: 10  20  30  
top = 2
```

Test:

```
peek(st, 1);  
peek(st, 2);  
peek(st, 3);  
peek(st, 4);  
peek(st, 0);
```

Expected results:

```
peek(1) = 30  
peek(2) = 20  
peek(3) = 10  
peek(4) = Invalid Index  
peek(0) = Invalid Index
```

56. Final Lesson 4 Summary

Lesson 4 teaches how to build a complete C array-stack program.

The stack uses this structure:

```
struct Stack {  
    int size;  
    int top;  
    int *S;  
};
```

The stack is empty when:

```
top == -1
```

The stack is full when:

```
top == size - 1
```

`create()` initializes the stack and allocates memory.

`display()` prints from `top` down to `0`.

`push()` inserts a value at the top.

`pop()` removes and returns the top value.

`peek(pos)` reads a value at a position from the top.

`stackTop()` reads the current top element.

`isEmpty()` checks whether the stack is empty.

`isFull()` checks whether the stack is full.

Functions that modify the stack use pointers:

```
create(&st);  
push(&st, value);  
pop(&st);
```

Functions that only read the stack can receive it by value:

```
display(st);  
peek(st, pos);  
stackTop(st);  
isEmpty(st);  
isFull(st);
```

The most important idea in Lesson 4 is:

A complete array-stack program is built by correctly controlling `top`.

If `top` is correct, the stack works correctly.

If `top` is wrong, the whole stack behavior becomes wrong.

So always track:

```
What is top before the operation?  
What is top after the operation?  
Which array indices are active?  
Which values are only old physical memory and no longer part of the logical  
stack?
```

That is the core skill of Lesson 4.